

Manuale Tecnico – The-Knife

Versione: 2.0 • Data: 20/08/2026

Autori:

Francesco Girlanda 760616 VA

Mattia Lambertoni 762595 VA

Gabriele Gallon 761125 VA

Sommario

Introduzione.....	3
Architettura del Sistema.....	3
Architettura Generale.....	3
Moduli del Progetto.....	4
Progettazione.....	6
Diagramma delle Classi - Sistema Completo.....	6
Diagramma di Sequenza - Login.....	7
Diagramma di Sequenza - Aggiunta Recensione.....	8
Diagramma di Sequenza - Avvio Server.....	9
Scelte Architettureali.....	10
1. Architettura Client-Server.....	10
2. RMI per la Comunicazione.....	10
3. JavaFX per la GUI.....	10
Design Patterns.....	11
1. Data Access Object (DAO) Pattern.....	11
Mappatura dei risultati.....	11
Operazioni tipiche.....	11
Gestione delle risorse e delle eccezioni.....	11
2. Service Layer Pattern.....	12
3. Model-View-Controller (MVC).....	13
Strutture Dati.....	14
Collezioni standard.....	14
Modelli di dominio.....	14
Utente.....	14
Ristorante.....	14
Recensione.....	14
Preferito.....	15
Sessione.....	15
Coordinate.....	15
Enum come tipi controllati.....	15
Database.....	16
Analisi dei Requisiti.....	16
Requisiti funzionali.....	16
Diagrammi ER.....	17
Diagramma ER non ristrutturato.....	17
Ristrutturazione ER.....	18
Vincoli e Relazioni.....	19
Schema Relazionale.....	19
Stack Tecnologico.....	20
Bibliografia e riferimenti.....	21

Introduzione

The-Knife è un'applicazione Java con interfaccia **JavaFX** per la consultazione e gestione di ristoranti. I dati vengono memorizzati su un **database** e l'interfaccia è definita in **FXML** tramite l' utilizzo di **SceneBuilder**. La struttura del progetto è realizzata con **Maven**

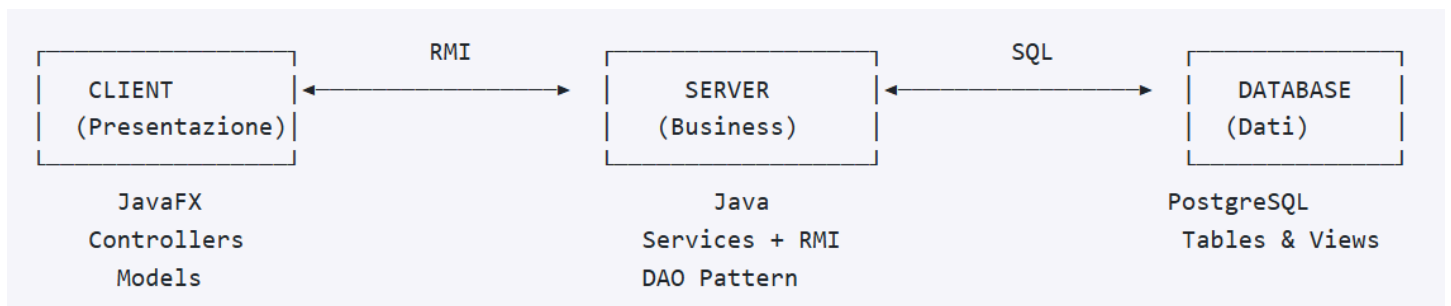
Il progetto comprende:

- Utilizzo di pattern architetturali (DAO, MVC)
- Comunicazione remota tramite RMI (Remote Method Invocation)
- Persistenza dei dati su database relazionale PostgreSQL
- Interfaccia grafica moderna con JavaFX
- Gestione di autenticazione e autorizzazione
- Implementazione di operazioni CRUD complete

Architettura del Sistema

Architettura Generale

The Knife segue un'architettura client-server multi-tier divisa in tre componenti principali:



Moduli del Progetto

1. Client Module (client/):

- Interfaccia utente sviluppata in JavaFX
- Controllers per la gestione della logica di presentazione
- Comunicazione con il server tramite RMI
- Struttura:

```
client/
├─ README.md
├─ pom.xml
├─ dependency-reduced-pom.xml
├─ src/main/java/com/gruppo10/
│   ├─ controller/                # Controller JavaFX
│   ├─ gui_elements/              # Utility grafica
│   ├─ ClientContext.java/        # Contesto client
│   ├─ ClientTK.java/             # Applicazione
│   └─ ClientBoot.java            # Entry point
└─ src/main/resources/
    ├─ images                     # Immagini GUI
    └─ GUI/                       # FXML (interfacce)
```

2. Server Module (server/):

- Interfaccia utente sviluppata in JavaFX
- Logica di business e servizi remoti
- Implementazione dei Service
- Data Access Objects (DAO) per accesso ai dati
- Autenticazione e autorizzazione
- Gestione della connessione al database
- Struttura:

```
server/
├─ README.md
├─ pom.xml
├─ dependency-reduced-pom.xml
├─ src/main/java/com/gruppo10/
│   ├─ servizi_imp/               # Servizi remoti
│   ├─ database/                  # Data Access Objects
│   ├─ controller/                # Controller JavaFX
│   ├─ permessi/                  # Gestione permessi
│   ├─ ServerBoot.java            # Entry point
│   ├─ ServerContext.java         # Contesto server
│   ├─ ServerPublisher.java       # Pubblicazione dei servizi
│   └─ ServerTK.java              # Applicazione
└─ src/main/resources/
    ├─ images                     # Immagini GUI
    └─ GUI/                       # Pannello amministrativo
```

3. Common Module (common/):

- Classi condivise tra client e server
- Data Transfer Objects (DTO)
- Interfacce remote dei servizi per RMI
- Eccezioni comuni
- Struttura:

```
common/  
├─ README.md  
├─ pom.xml  
├─ src/main/java/com/gruppo10/  
│   ├─ classi/           # Modelli comuni  
│   ├─ servizi_int/      # Interfacce remote  
│   └─ eccezioni/        # Eccezioni comuni
```

Progettazione

Diagramma delle Classi - Sistema Completo

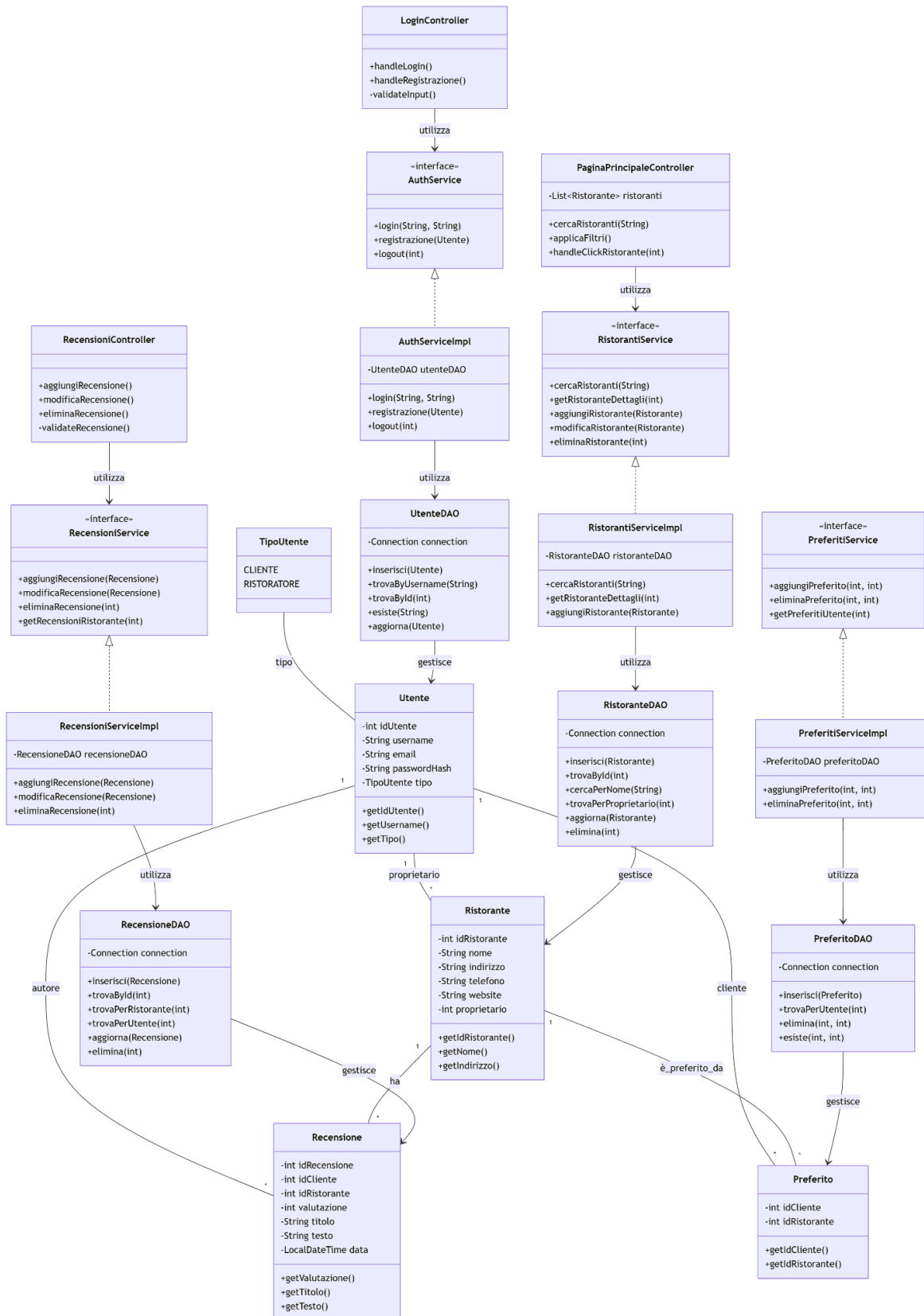


Diagramma di Sequenza - Login

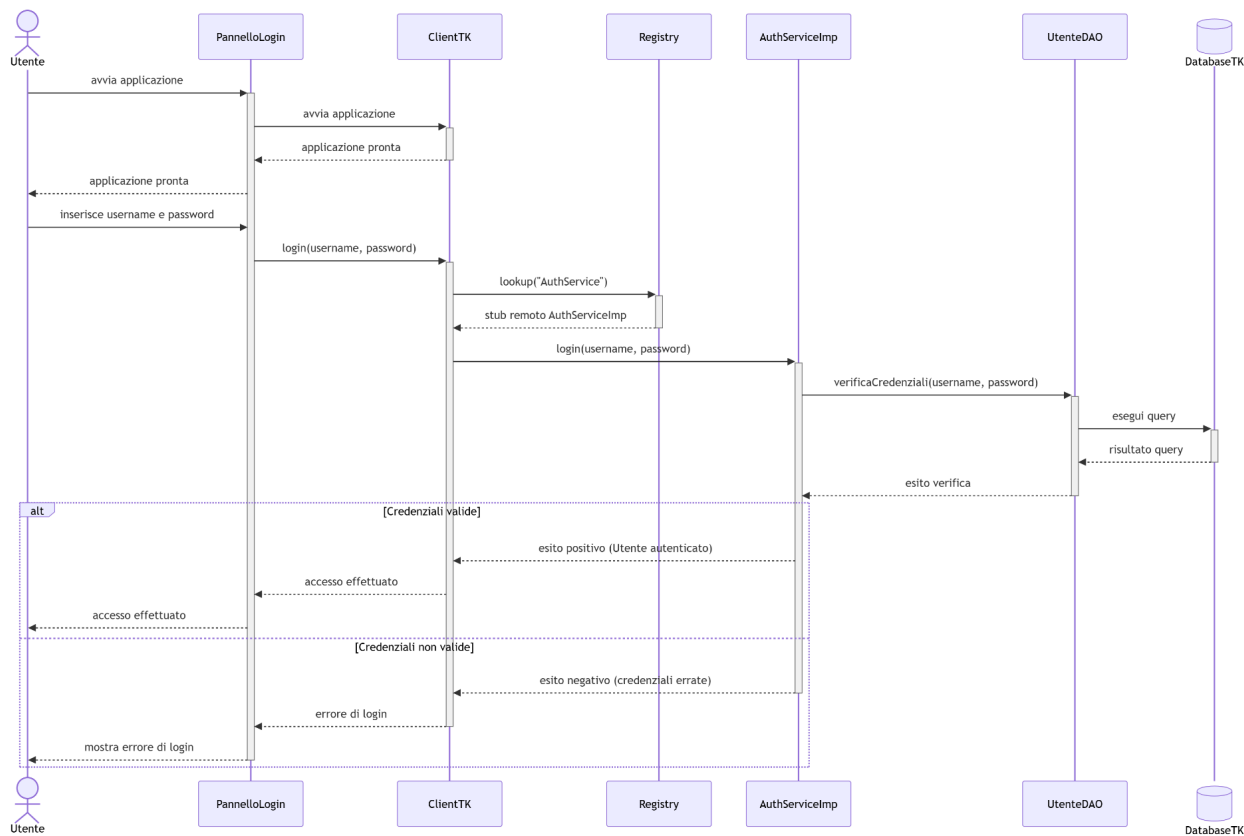
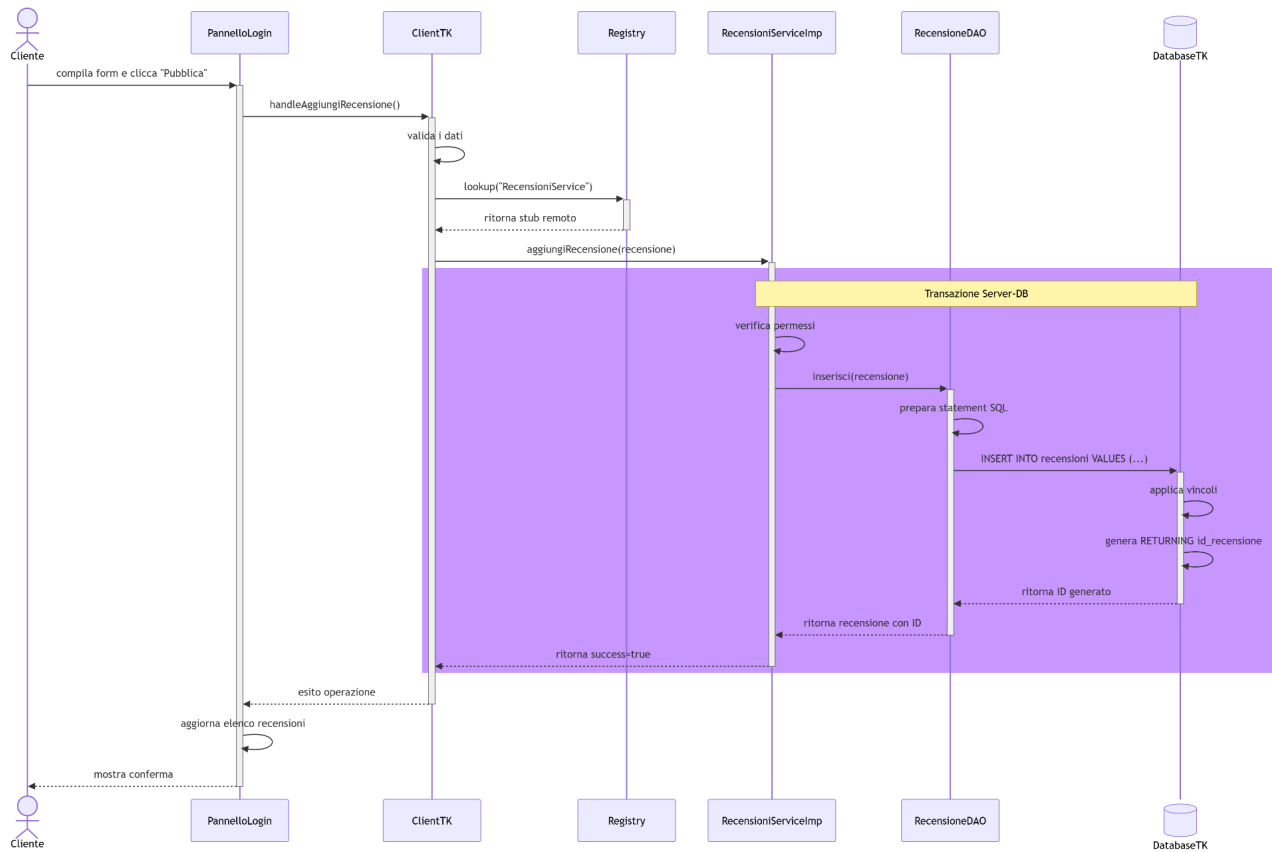
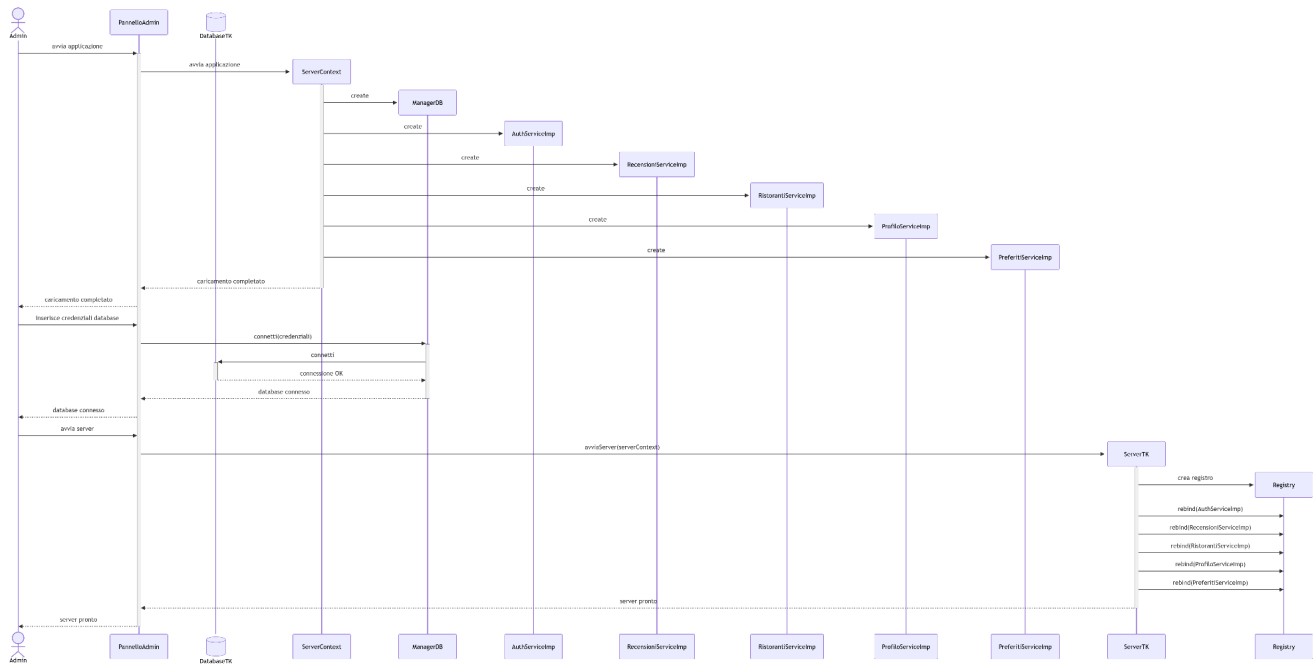


Diagramma di Sequenza - Aggiunta Recensione



Nota: il diagramma sopra riportato rimane valido anche per le altre operazioni, ovvero: aggiunta e rimozione dei preferiti, rimozione e modifica delle recensioni, aggiunta di risposta alle recensioni.

Diagramma di Sequenza - Avvio Server



Scelte Architettureali

1. Architettura Client-Server

Divisione in 3 moduli Maven separati all'interno dello stesso progetto, l'interazione tra moduli è gestita da Maven tramite i file pom.xml .

- **Modulo client:** applicazione lato client
- **Modulo server:** applicazione lato server
- **Modulo common:** elementi condivisi tra client e server

2. RMI per la Comunicazione

Motivazione:

- Invocazione remota di metodi Java
- Serializzazione automatica dei dati
- Gestione semplificata della comunicazione
- Supporto nativo per Java

3. JavaFX per la GUI

Motivazione:

- Moderna e ricca di componenti
- Supporto completo per CSS e animazioni
- Tooling completo (SceneBuilder)
- Integrata in Java moderno

Design Patterns

1. Data Access Object (DAO) Pattern

Ogni entità del dominio ha un corrispondente DAO che gestisce l'accesso ai dati. Il pattern DAO isola la logica di business dal livello di persistenza: le classi di dominio non contengono alcun riferimento a SQL o JDBC, e il database potrebbe in linea di principio essere sostituito senza impattare il resto dell'applicazione.

Per evitare di duplicare in ogni DAO la logica di apertura connessione ed esecuzione query, tutte le classi DAO estendono una superclasse astratta comune, che centralizza le operazioni ricorrenti: selezione di una riga o di una lista di righe, selezione di un valore scalare (es. un **EXISTS** o un **AVG**), esecuzione di **INSERT/UPDATE/DELETE**, e inserimento con recupero dell'ID generato dal database. Ogni sottoclasse si limita così a scrivere lo SQL specifico della propria entità e la relativa logica di mappatura, senza gestire direttamente connessioni e statement.

Mappatura dei risultati

La traduzione da riga del database a oggetto Java è affidata a un'interfaccia funzionale, passata come parametro ai metodi di selezione: questo disaccoppia la logica di query (comune) da quella di mappatura (specifica per entità). Per query più complesse, in cui un'entità è composta da più righe (es. un'entità con una lista di elementi correlati ottenuta tramite **JOIN**), il singolo DAO può definire una logica di aggregazione dedicata.

Operazioni tipiche

I DAO coprono tipicamente il ciclo di vita CRUD dell'entità: l'inserimento avviene tramite **PreparedStatement** parametrizzato — per prevenire SQL Injection — previa validazione dei dati obbligatori; il recupero avviene tramite query **SELECT** filtrate sugli attributi rilevanti, restituendo un **Optional** vuoto o una lista vuota in assenza di risultati (mai **null**); aggiornamento ed eliminazione restituiscono il numero di righe modificate come conferma dell'esito.

Gestione delle risorse e delle eccezioni

Ogni metodo che accede al database opera all'interno di un blocco **try-with-resources**, garantendo il rilascio automatico di connessione, statement e result set, a prescindere dall'esito dell'operazione. Le eccezioni SQL generiche vengono propagate al chiamante; le violazioni di vincoli di unicità (es. un valore duplicato) vengono invece tradotte in eccezioni applicative dedicate, più significative per la gestione dell'errore ai livelli superiori.

2. Service Layer Pattern

I servizi implementano la logica di business dell'applicazione e vengono esposti ai client distribuiti tramite l'architettura Java RMI (Remote Method Invocation). Ogni funzionalità del sistema è organizzata secondo lo schema **interfaccia remota + implementazione**, seguendo un pattern comune a tutti i service.

La struttura e le responsabilità del codice si articolano nei seguenti punti chiave:

- **Contratto Remoto (interfaccia `*ServiceInt`):** Ogni area funzionale del sistema (autenticazione, ristoranti, recensioni, profilo, preferiti, ecc.) è definita da un'interfaccia dedicata che estende `Remote`. Ogni firma di metodo dichiara la possibile propagazione di una `RemoteException`, requisito fondamentale per gestire gli errori di rete nella comunicazione client-server in RMI.
- **Implementazione Server (`*ServiceImp`):** Le classi di implementazione estendono la classe base `BasicServiceImp`, che estende `UnicastRemoteObject` e mette a disposizione l'accesso condiviso a `ManagerDB` (per il livello dati) e `SessionManager` (per la gestione delle sessioni attive). Ogni service delega le operazioni di accesso ai dati al DAO specifico ottenuto tramite `ManagerDB`, mantenendo la logica di business separata dalla persistenza.
- **Controllo degli Accessi:** Le operazioni che richiedono un particolare ruolo utente (es. azioni riservate a un cliente registrato o a un ristoratore) vengono precedute da una verifica dei permessi tramite il token di sessione, effettuata con il metodo `verificaPermessi` ereditato da `BasicServiceImp`. Se il chiamante non possiede il ruolo necessario, viene sollevata una `PermessoNegatoException`; le operazioni di sola lettura o non riservate, invece, non richiedono questa verifica.
- **Gestione degli Errori e Information Hiding:** Ogni metodo cattura le eventuali `SQLException` sollevate dal livello DAO durante l'accesso al database e le converte in una `RemoteException` con un messaggio descrittivo dell'operazione fallita. Questo evita di esporre ai client dettagli sensibili sull'infrastruttura del database (query, driver, struttura delle tabelle) in caso di errore, propagando verso l'esterno solo un'eccezione di alto livello coerente con il contratto RMI. Eventuali eccezioni applicative specifiche (come `UsernameGiaEsistenteException` o `PermessoNegatoException`) vengono invece rilanciate così come sono, per permettere al client di distinguere un errore di dominio da un errore infrastrutturale.

3. Model-View-Controller (MVC)

I controller JavaFX gestiscono l'interazione tra la UI e i servizi remoti: ogni schermata dell'applicazione client è associata a una classe controller che funge da intermediario (secondo il pattern architetturale MVC) tra le interazioni dell'utente sulla vista e la logica di business fornita dal server remoto tramite RMI.

La logica dei controller si sviluppa attraverso i seguenti elementi chiave:

- **Binding dei Componenti Visivi (@FXML):** Le annotazioni collegano direttamente gli elementi definiti nel file di layout (vista) alle variabili Java del controller. Questo permette di interagire con i campi di input, le liste, i bottoni e le aree dedicate ai messaggi di feedback (es. label di errore), mantenendo la vista sincronizzata con lo stato gestito dal controller.
- **Invocazione dei Servizi RMI:** Sfruttando le interfacce remote (**ServiceInt*) recuperate tramite lookup, il controller inoltra le richieste dell'utente al server tramite chiamate remote — ad esempio l'invio di credenziali, la ricerca di ristoranti, l'inserimento di una recensione — passando eventualmente il token di sessione dell'utente autenticato quando l'operazione lo richiede.
- **Navigazione e Gestione dello Stato:** In caso di esito positivo dell'operazione, il controller aggiorna lo stato globale dell'applicazione client quando necessario (ad esempio salvando l'utente corrente o i dati recuperati) e procede con l'azione conseguente prevista dal flusso applicativo, tipicamente il caricamento di una nuova schermata o l'aggiornamento della vista corrente con i dati ottenuti.
- **Gestione Controllata degli Errori:** In caso di fallimento dell'operazione (es. credenziali errate, permessi insufficienti, dati non validi), il controller mostra all'utente un messaggio di errore appropriato tramite gli elementi della UI dedicati al feedback. Inoltre, ogni chiamata remota è racchiusa in un blocco try-catch che intercetta la *RemoteException*: questo garantisce che, qualora il server sia irraggiungibile o si verificano problemi di rete, l'applicazione non vada in crash ma fornisca all'utente un avviso chiaro (es. "Errore di connessione al server"), preservando la stabilità del client anche in condizioni di malfunzionamento del backend.

Strutture Dati

Il modello dati del sistema si basa su una combinazione di collezioni standard Java e classi domain-specific, scelte per rappresentare in modo chiaro e stabile le entità del dominio e i loro rapporti.

Collezioni standard

- **ArrayList**: impiegata per contenere insiemi di oggetti, come le recensioni associate a un ristorante o i risultati restituiti dalle query di ricerca.
- **LinkedHashMap**: usata in fase di mappatura dei dati del database, per aggregare righe di `ResultSet` in oggetti `Ristorante` preservando l'ordine di lettura.
- **ConcurrentHashMap**: adottata in `SessionManager` per tenere traccia delle sessioni attive in ambiente concorrente, garantendo accesso thread-safe a token e utenti autenticati.
- **Optional**: usato per rappresentare l'assenza di risultati nelle query di selezione, evitando valori `null` espliciti.
- **StringBuilder**: impiegato per costruire dinamicamente le query SQL in base ai filtri applicati, rendendo flessibile la ricerca dei ristoranti.

Modelli di dominio

Accanto alle collezioni standard, il dominio applicativo è rappresentato da classi custom nel ruolo di **Data Model (POJO)**: incapsulano lo stato delle entità e ne permettono il trasferimento tra i vari livelli del sistema (database, server RMI, client JavaFX). Tutte seguono lo stesso principio di incapsulamento: campi dichiarati `private`, esposti in lettura (e, dove necessario, in scrittura) esclusivamente tramite Getter e Setter.

Utente

Rappresenta l'account di un utilizzatore del sistema. Memorizza la chiave primaria autogenerata dal database (`idUtente`) insieme ai dati anagrafici di base (`username`, `email`). Per motivi di sicurezza, la password non è mai conservata né trasportata in chiaro: viene mantenuta esclusivamente nella sua forma cifrata (`passwordHash`). Il ruolo dell'account è definito tramite l'enumerativo `TipoUtente` (es. `CLIENTE` o `RISTORATORE`), attributo essenziale per il controllo degli accessi (RBAC) e per l'abilitazione delle relative funzionalità nell'interfaccia grafica.

Ristorante

Rappresenta l'entità ristorante. Raccoglie le informazioni descrittive e di contatto (nome, indirizzo, telefono, website) insieme alle coordinate geografiche (latitudine, longitudine), necessarie per eventuali funzionalità di visualizzazione su mappa. L'attributo proprietario funge da chiave esterna logica verso `Utente`, associando in modo univoco ogni ristorante all'account che lo gestisce.

Recensione

Rappresenta il feedback lasciato da un cliente su un ristorante. Gli attributi `idCliente` e `idRistorante` fungono da chiavi esterne logiche, collegando la recensione rispettivamente all'autore e alla struttura recensita. Il contenuto della valutazione unisce un giudizio quantitativo (valutazione, su scala numerica da 1 a 5) e uno qualitativo testuale (titolo, testo). La data e l'orario di pubblicazione sono tracciati tramite un oggetto `LocalDateTime`.

Preferito

Rappresenta la relazione tra un cliente e un ristorante salvato tra i preferiti, mappando una tabella di associazione che risolve la relazione molti-a-molti tra le due entità. A differenza delle altre classi di dominio, non utilizza un identificativo singolo: la chiave primaria è composta dalla coppia idCliente/idRistorante, il che impedisce a un utente di salvare lo stesso ristorante tra i preferiti più di una volta. Entrambi i campi operano come chiavi esterne logiche verso [Utente](#) e [Ristorante](#).

Sessione

Rappresenta una sessione utente autenticata, generata al momento del login e utilizzata per riconoscere le richieste successive senza richiedere nuovamente le credenziali. Associa un token identificativo univoco all'utente autenticato, permettendo al sistema di verificare la validità dell'accesso a ogni operazione protetta. Le sessioni attive sono gestite da [SessionManager](#) tramite una [ConcurrentHashMap](#) (vedi sezione precedente), che ne garantisce la consultazione e la modifica in sicurezza anche in presenza di più richieste concorrenti.

Coordinate

Rappresenta un punto geografico tramite la coppia di valori latitudine/longitudine. È utilizzata sia dall'entità [Ristorante](#), per la posizione del locale, sia per la posizione dell'utente nelle ricerche basate sulla distanza. Fornisce il supporto necessario alla geolocalizzazione e al calcolo delle distanze tra due punti, funzionalità sfruttata dai filtri di ricerca dei ristoranti.

Enum come tipi controllati

Il sistema impiega inoltre enum per modellare valori controllati e semantici del dominio, in sostituzione di stringhe o costanti numeriche arbitrarie:

- **Ruolo**: definisce il tipo di account (es. cliente o ristoratore), usato per il controllo degli accessi.
- **TipoCucina**: definisce le categorie di cucina disponibili per un ristorante, usato sia in fase di creazione sia come filtro di ricerca.
- **Prezzo**: definisce le fasce di prezzo di un ristorante, usato come filtro di ricerca con soglia massima.
- **Delivery**: definisce la disponibilità del servizio di consegna a domicilio.
- **Prenotazione**: definisce la disponibilità della prenotazione online.
- **MediaRecensioni**: definisce le soglie di voto medio utilizzabili come filtro minimo nella ricerca.
- **Distanza**: definisce le soglie di distanza utilizzabili come filtro nella ricerca geolocalizzata.

L'uso di enum evita valori arbitrari o incoerenti, rende la logica di filtro più robusta (i confronti avvengono su un insieme chiuso di valori noti a compile-time) e migliora la leggibilità sia del codice sia dell'interfaccia utente, dove i valori possono essere mappati direttamente su etichette testuali o elementi di selezione.

Database

Analisi dei Requisiti

La progettazione della base di dati ha lo scopo di rappresentare e gestire in modo persistente le informazioni utilizzate dall'applicazione The Knife.

L'applicazione consente agli utenti di registrarsi e autenticarsi, cercare ristoranti, applicare filtri di ricerca, visualizzare le informazioni e le recensioni di un ristorante, salvare ristoranti tra i preferiti e pubblicare recensioni. Gli utenti con ruolo ristoratore possono inoltre inserire ristoranti e rispondere alle recensioni ricevute.

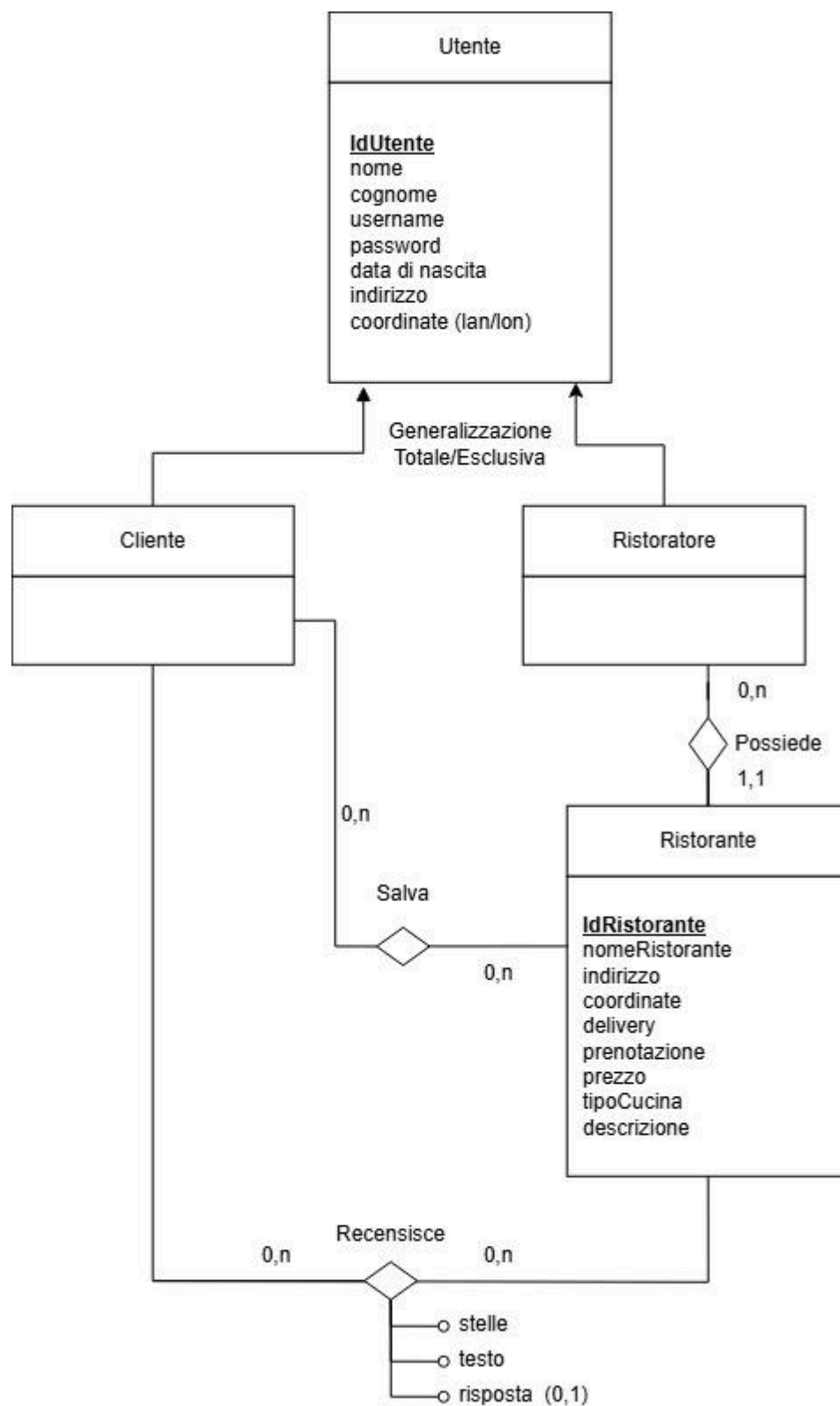
Requisiti funzionali

La base di dati deve supportare le seguenti operazioni:

- Memorizzare gli utenti registrati, distinguendoli mediante un ruolo ed uno username univoco.
- Memorizzare i ristoranti e identificarli univocamente.
- Memorizzare i preferiti come associazioni tra utenti e ristoranti.
- Memorizzare le recensioni e identificarle univocamente.
- Associare ogni ristorante ad un utente proprietario esistente.
- Associare ogni recensione ad un utente e a un ristorante esistenti.
- Associare ogni preferito ad un utente e ad un ristorante esistenti.

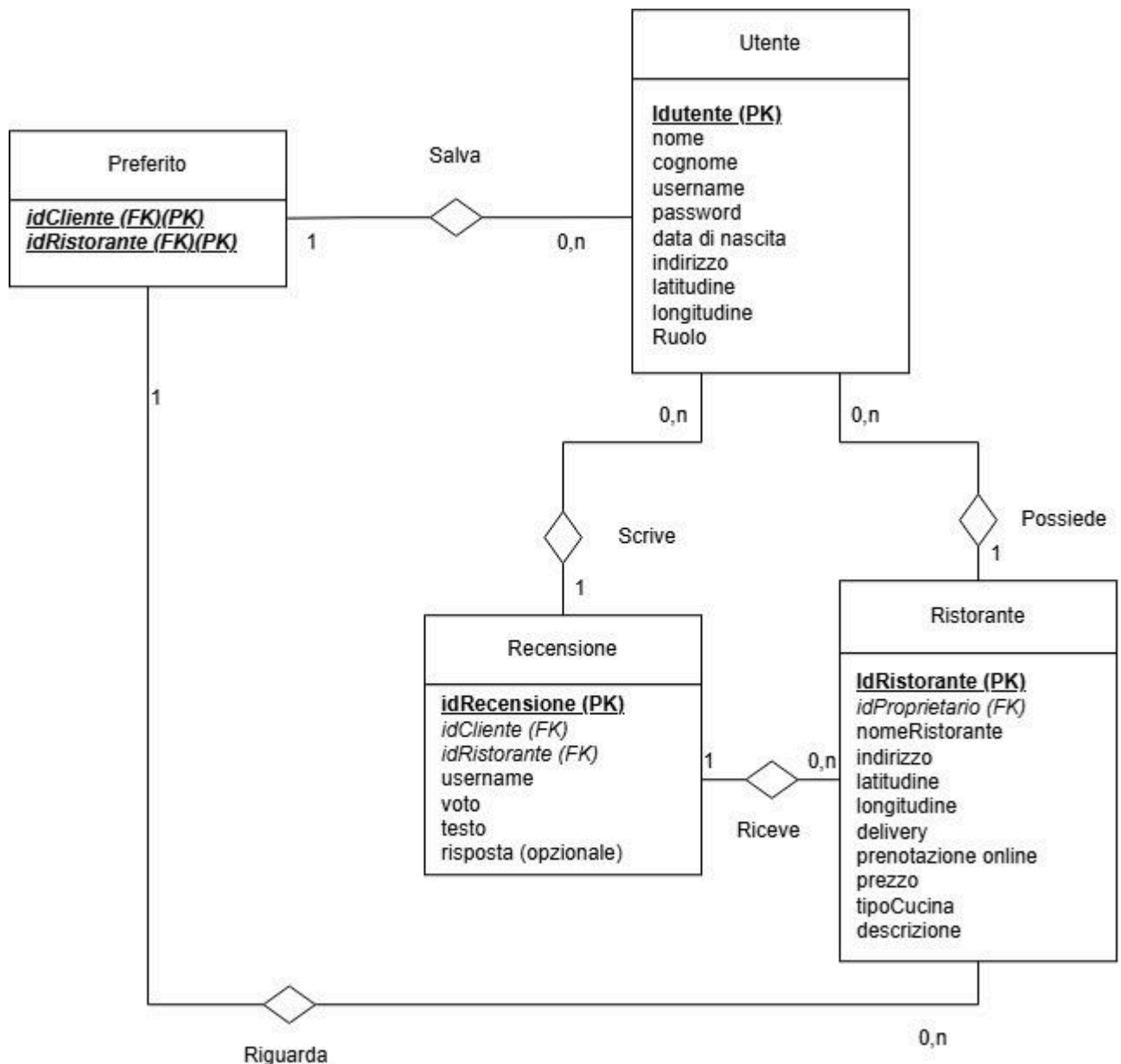
Diagrammi ER

Diagramma ER non ristrutturato



Ristrutturazione ER

- **Eliminazione delle generalizzazioni:**
 - visto che le entità Cliente e ristorante non possiedono attributi propri, la generalizzazione è stata eliminata accorpendo le specializzazioni nell'entità padre (Utente) introducendo un nuovo attributo "Ruolo"
- **Scomposizione attributi complessi:**
 - l'attributo composto "Coordinate" è stato scomposto nei due attributi atomici "Latitudine" e "Longitudine"
- **Conversione associazioni multi-a-molti:**
 - l'associazione "Recensisce" con i suoi attributi è stata trasformata in nell'entità "Recensione" collegata ad "Utente" e "Ristorante" mediante due associazioni uno-a-molti
 - l'associazione "Salva" è stata trasformata nell'entità "Preferito" collegata ad "Utente" e "Ristorante" mediante due associazioni uno-a-molti



Vincoli e Relazioni

1. Vincolo di Unicità su username:

- Evita la registrazione di utenti con lo stesso username
- Implementato a livello database

2. Una recensione per cliente-ristorante:

- Chiave composta UNIQUE su (id_cliente, id_ristorante)
- Ogni cliente può scrivere una sola recensione per ristorante

3. Preferiti con chiave composta:

- La coppia (id_cliente, id_ristorante) è univoca
- Previene duplicati nella lista dei preferiti

Schema Relazionale

Utente: (**Id Utente**, nome,cognome, Username, password, data di nascita, indirizzo, latitudine, longitudine, Ruolo)

Ristorante: (**IdRistorante**, idProprietario^{Utente}, nomeRistorante, indirizzo, latitudine, longitudine, delivery, prenotazioneOnline, prezzo, tipoCucina,descrizione)

Recensione: (**IdRecensione**, idCliente^{Utente}, idRistorante^{Ristorante}, username, voto, testo, risposta)

Preferito: (**idCliente**^{Utente}, **idRistorante**^{Ristorante})

Stack Tecnologico

Componente	Tecnologia	Versione	Ruolo
Linguaggio	Java	24.0.2	Linguaggio di programmazione
GUI Client	JavaFX	24.0.1	Framework interfaccia grafica
Build Tool	Apache Maven	3.9.9	Gestione dipendenze e build
Database	PostgreSQL	18.4	DBMS relazionale
Comunicazione	Java RMI	Native	Protocol remoto
Testing	JUnit	4.13.2	Framework testing
Logging	SLF4J + Logback	1.7.36	Logging strutturato
Connection Pool	HikariCP	5.0.1	Pool di connessioni database
Password Hashing	BCrypt	0.9.1	Hash sicuro password

Bibliografia e riferimenti

- JavaFX (FXML, Controls): [*OpenJFX docs*](#)
- Maven: [*Maven Guides*](#)
- OpenCSV: [*opencsv user guide*](#)
- Gson: [*Gson javadoc*](#)
- OpenStreetMap Nominatim: [*manual*](#)